

HAND-ON LAB: KALKULATOR DESKTOP DENGAN C#

Windows Forms App • .NET • Visual Studio

Mata Kuliah	NET Programming dengan C#
Studi Kasus	Calculator Desktop
Level	Pemula – Menengah
Durasi	± 120 menit

1. Tujuan Pembelajaran

- Membuat project Windows Forms App dengan C#.
- Mendesain form menggunakan Visual Designer.
- Menggunakan Label, TextBox, dan Button.
- Menangani event Click dengan event handler.
- Menerapkan variable, method, operator aritmatika, if/switch.
- Melakukan validasi input dan menangani error dengan try-catch.

2. Prasyarat & Tools

- Windows + Visual Studio yang mendukung Windows Forms dan .NET.
- Pemahaman dasar C#: variable, method, operator, dan if/switch.
- Tidak memerlukan database untuk lab dasar ini.

3. Langkah Praktikum

Langkah 1 — Membuat Project

- Buka Visual Studio.
- Pilih Create a new project.
- Pilih template Windows Forms App.
- Language: C#.
- Framework: .NET.
- Nama project: CalculatorApp.
- Klik Create.

Hasil: Visual Studio membuat Form1 dan file project otomatis.

```
CalculatorApp
|
├─ Program.cs
├─ Form1.cs
├─ Form1.Designer.cs
└─ Form1.resx
```

Langkah 2 — Mendesain Form

- Buka Toolbox → Windows Forms.
- Tambahkan Label, TextBox, dan Button ke Form1.
- Atur Name dan Text sesuai tabel berikut.

Control	Name	Text
Label	lblTitle	Calculator
TextBox	txtDisplay	0
Button	btn7	7
Button	btn8	8
Button	btn9	9
Button	btnDivide	÷
Button	btn4	4
Button	btn5	5
Button	btn6	6
Button	btnMultiply	×
Button	btn1	1
Button	btn2	2
Button	btn3	3
Button	btnMinus	−
Button	btn0	0
Button	btnDecimal	.
Button	btnClear	C
Button	btnPlus	+
Button	btnEquals	=

Tip desain: Susun tombol dalam pola 4 kolom. Besarkan txtDisplay dan gunakan TextAlign = MiddleRight.

Langkah 3 — Menyiapkan Variabel

```
double firstNumber = 0;  
double secondNumber = 0;  
double result = 0;  
  
string operation = "";
```

Konsep: firstNumber menyimpan angka pertama, operation menyimpan operator, secondNumber menyimpan

angka kedua, result menyimpan hasil.

Langkah 4 — Event Tombol Angka

```
private void NumberButton_Click(object sender, EventArgs e)
{
    Button button = (Button)sender;

    if (txtDisplay.Text == "0")
        txtDisplay.Text = button.Text;
    else
        txtDisplay.Text += button.Text;
}
```

- Pilih semua tombol angka (0–9).
- Pada Properties → Events → Click, arahkan ke NumberButton_Click.
- Dengan satu handler, kode tidak perlu diulang untuk setiap tombol.

Langkah 5 — Event Tombol Operator

```
private void OperatorButton_Click(object sender, EventArgs e)
{
    Button button = (Button)sender;

    firstNumber = double.Parse(txtDisplay.Text);
    operation = button.Text;

    txtDisplay.Clear();
}
```

Contoh: Saat user memilih 10 lalu +, maka firstNumber = 10 dan operation = "+".

Langkah 6 — Menghitung Hasil dengan switch

```
private void btnEquals_Click(object sender, EventArgs e)
{
    try
    {
        secondNumber = double.Parse(txtDisplay.Text);

        switch (operation)
        {
            case "+": result = firstNumber + secondNumber; break;
            case "-": result = firstNumber - secondNumber; break;
            case "x": result = firstNumber * secondNumber; break;
            case "÷":
                if (secondNumber == 0)
                    throw new DivideByZeroException();
                result = firstNumber / secondNumber;
                break;
        }

        txtDisplay.Text = result.ToString();
    }
    catch (Exception ex)
    {
    }
```

```

        MessageBox.Show(ex.Message, "Error");
    }
}

```

Langkah 7 — Clear & Decimal

```

private void btnClear_Click(object sender, EventArgs e)
{
    firstNumber = 0;
    secondNumber = 0;
    result = 0;
    operation = "";
    txtDisplay.Text = "0";
}

private void btnDecimal_Click(object sender, EventArgs e)
{
    if (!txtDisplay.Text.Contains("."))
        txtDisplay.Text += ".";
}

```

Langkah 8 — Menguji Aplikasi

Skenario	Input	Expected Result	Status
Penjumlahan	$10 + 20 =$	30	✓ Berhasil
Pengurangan	$30 - 12 =$	18	✓ Berhasil
Perkalian	$6 \times 7 =$	42	✓ Berhasil
Pembagian	$100 \div 4 =$	25	✓ Berhasil
Desimal	$2.5 \times 4 =$	10	✓ Berhasil
Bagi nol	$10 \div 0 =$	Pesan error	✓ Berhasil
Clear	Tekan C	Display = 0	✓ Berhasil

4. Kerangka Form1.cs

```

namespace CalculatorApp
{
    public partial class Form1 : Form
    {
        double firstNumber = 0;
        double secondNumber = 0;
        double result = 0;
        string operation = "";

        public Form1()
        {
            InitializeComponent();
        }
    }
}

```

```

        // NumberButton_Click(...)
        // OperatorButton_Click(...)
        // btnEquals_Click(...)
        // btnClear_Click(...)
    }
}

```

5. Tantangan Pengembangan

- Tambahkan tombol $\pm$ dan $\%$.
- Tambahkan Backspace.
- Tambahkan riwayat perhitungan.
- Pisahkan logika kalkulator ke class/service terpisah.
- Kembangkan menjadi Scientific Calculator.

6. Refleksi Mahasiswa

☐ Apa fungsi object sender pada event handler?

Jawaban: Parameter sender bertipe object merepresentasikan referensi ke kontrol antarmuka (UI) yang memicu terjadinya event (misalnya tombol yang sedang diklik). Dengan melakukan type casting ke tipe kontrol yang sesuai (contoh: `Button button = (Button)sender;`), program dapat mengakses properti kontrol tersebut secara dinamis seperti `button.Text` atau `button.Name`. Hal ini memungkinkan satu method event handler digunakan bersama oleh banyak tombol secara efisien.

☐ Mengapa semua tombol angka dapat memakai satu `NumberButton_Click`?

Jawaban: Semua tombol angka (0-9) memiliki logika pemrosesan yang identik, yaitu menambahkan digit angka dari tombol yang ditekan ke `TextBox` tampilan (atau menggantikan teks "0" default saat kondisi awal). Melalui parameter sender, method `NumberButton_Click` dapat mengetahui secara dinamis angka mana yang ditekan via `button.Text`. Penggunaan satu method bersama ini menerapkan prinsip DRY (Don't Repeat Yourself), mengeliminasi redundansi 10 method terpisah, serta mempermudah pemeliharaan kode program.

☐ Apa perbedaan `firstNumber`, `secondNumber`, dan `result`?

Jawaban: `firstNumber` berfungsi menyimpan nilai angka pertama sebelum pengguna memilih tombol operator aritmatika (+, -, x, /). `secondNumber` menyimpan nilai angka kedua yang diinput pengguna setelah operator dan sebelum tombol hitung (=) ditekan. Sedangkan `result` menampung hasil evaluasi matematis antara `firstNumber` dan `secondNumber` sesuai operasi yang dipilih, yang kemudian dikonversi menjadi teks string untuk ditampilkan ke layar kalkulator (`txtDisplay`).

☐ Mengapa pembagian dengan nol perlu divalidasi?

Jawaban: Secara matematis, operasi pembagian dengan nilai nol tidak terdefinisi. Dalam C# dengan tipe data `double` (standar IEEE 754), operasi pembagian angka dengan nol tidak melempar runtime exception melainkan menghasilkan nilai khusus `Infinity` atau `NaN`, yang dapat merusak keakuratan komputasi dan membingungkan pengguna. Validasi eksplisit `if (secondNumber == 0) throw new DivideByZeroException();` memastikan kondisi tersebut ditangani secara elegan dan memunculkan dialog pesan error yang ramah bagi pengguna.

☐ Bagaimana try-catch membantu menjaga aplikasi tetap stabil?

Jawaban: Blok try-catch menerapkan mekanisme penanganan eksepsi terstruktur (Structured Exception Handling). Kode yang berisiko memicu kesalahan runtime (seperti kegagalan `double.Parse` saat format teks tidak valid atau `DivideByZeroException` saat pembagian nol) ditempatkan di dalam blok try. Ketika terjadi runtime error, eksekusi dicegat oleh blok catch sehingga aplikasi tidak mengalami force close (crash), melainkan menampilkan `MessageBox` pesan kesalahan yang informatif dan tetap stabil berjalan.